To kickstart your long-form **Creatorize** clone right now, **Option 2** is the best choice because it instantly generates the core source code files needed to get your local rendering engine and audio alignment pipeline running.
Here is the complete core file structure and implementation code to bring your local multi-agent video generator to life.

Step 1: Initialize the Project

Create a new directory for your engine, navigate into it, and initialize your project:

```bash
mkdir creatorize-engine
cd creatorize-engine
npm init -y
```

Update your package.json to support ES Modules and the required dependencies:

```json
{
  "name": "creatorize-engine",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node orchestrator.js"
  },
  "dependencies": {
    "fluent-ffmpeg": "^2.1.2",
    "three": "^0.160.0"
  }
}
```

Step 2: The Retro Three.js Rendering Core (renderer.js)

This module handles the Three.js scene setup. It implements the hard-pixel grid snappings required to mimic retro hardware limits.

JavaScript

```
import * as THREE from 'three';

export function createRetroEngine(width = 640, height = 360) {
  const scene = new THREE.Scene();
  scene.background = new THREE.Color(0x111118);

  const camera = new THREE.PerspectiveCamera(45, width / height, 0.1, 1000);
  camera.position.set(0, 5, 12);
  camera.lookAt(0, 0, 0);

  // Add ambient & directional retro lighting
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
  scene.add(ambientLight);

  const dirLight = new THREE.DirectionalLight(0xffffff, 0.8);
  dirLight.position.set(5, 10, 7);
  scene.add(dirLight);

  const retroMaterial = new THREE.MeshStandardMaterial({
    color: 0x3399ff,
    roughness: 0.8,
    metalness: 0.2,
    flatShading: true // Enhances the low-poly/retro aesthetic
  });

  return { scene, camera, retroMaterial };
}

export function updateSceneTime(mesh, timeInSeconds) {
  // Vertex snapping: quantizes movement to emulate old hardware grid constraints
  const gridStep = 0.05;
  mesh.position.x = Math.round(Math.sin(timeInSeconds) * 3 / gridStep) * gridStep;
  mesh.rotation.y = Math.floor(timeInSeconds * 30) / 30;
}
```

Step 3: The Headless FFmpeg Video Pipeline (renderer-core.js)

This file manages the rendering loop, generating raw frames and piping them straight into a local FFmpeg child process to output an encoded .mp4 video.

JavaScript

```
import { createRetroEngine, updateSceneTime } from './renderer.js';
import ffmpeg from 'fluent-ffmpeg';
import { PassThrough } from 'stream';
import * as THREE from 'three';

export async function renderChapterToDisk(outputPath, fps = 30, durationSec = 10) {
 const width = 640;
 const height = 360;
 const totalFrames = fps * durationSec;

 const { scene, camera, retroMaterial } = createRetroEngine(width, height);

 // Test mesh to represent a 3D asset or character model
 const geometry = new THREE.BoxGeometry(2, 2, 2);
 const cube = new THREE.Mesh(geometry, retroMaterial);
 scene.add(cube);

 // Setup headless software renderer (or standard WebGL fallback context)
 const renderer = new THREE.WebGLRenderer({ antialias: false, alpha: false,
preserveDrawingBuffer: true });
 renderer.setSize(width, height);

 const inputStream = new PassThrough();
 const ffmpegProcess = ffmpeg(inputStream)
  .inputFormat('rawvideo')
  .inputOptions([
   `-s ${width}x${height}`,
   `-pix_fmt rgba`,
   `-r ${fps}`
  ])
  .outputOptions([
   '-c:v libx264',
   '-pix_fmt yuv420p',
   '-crf 18',
   '-preset ultrafast'
  ])
  .output(outputPath);
```

```
  return new Promise((resolve, reject) => {
    ffmpegProcess.on('end', () => resolve(outputPath));
    ffmpegProcess.on('error', (err) => reject(err));
    ffmpegProcess.run();

    let currentFrame = 0;

    function writeNextFrame() {
      if (currentFrame >= totalFrames) {
        inputStream.end();
        return;
      }

      const time = currentFrame / fps;
      updateSceneTime(cube, time);

      renderer.render(scene, camera);

      // Extract pixel read buffer from WebGL canvas context
      const pixelBuffer = new Uint8Array(width * height * 4);
      const gl = renderer.getContext();
      gl.readPixels(0, 0, width, height, gl.RGBA, gl.UNSIGNED_BYTE, pixelBuffer);

      inputStream.write(pixelBuffer);

      currentFrame++;
      setImmediate(writeNextFrame);
    }

    writeNextFrame();
  });
}
```

Step 4: The Local Audio Transcription & Alignment Tool (align_audio.py)

Save this Python script to run local word-level timestamps using Whisper and a SciPy noise gate.

Python

```python
import sys
import json
import numpy as np
from scipy.io import wavfile
import whisper

def process_alignment(wav_path):
    sample_rate, data = wavfile.read(wav_path)
    if len(data.shape) > 1:
        data = data.mean(axis=1)

    # Basic noise gate threshold calculation
    abs_data = np.abs(data)
    threshold = np.max(abs_data) * 0.04
    active_speech = abs_data > threshold

    model = whisper.load_model("turbo")
    result = model.transcribe(wav_path, word_timestamps=True)

    timed_words = []
    for segment in result["segments"]:
        for word in segment["words"]:
            timed_words.append({
                "word": word["word"].strip(),
                "start": round(word["start"], 3),
                "end": round(word["end"], 3)
            })

    print(json.dumps({"success": True, "total_duration": len(data)/sample_rate, "words":
timed_words}, indent=2))

if __name__ == "__main__":
    process_alignment(sys.argv[1])
```

Step 5: The Master Orchestrator Script (orchestrator.js)

This script ties your rendering pipeline together, simulating how your multi-agent workflow triggers a local chapter compilation.

JavaScript

```
import { renderChapterToDisk } from './renderer-core.js';
import path from 'path';

async function runOrchestrator() {
 console.log('[Orchestrator] Initializing Creatorize Local Build Pipeline...');

 const outputFileName = 'chapter_01_output.mp4';
 const outputPath = path.resolve(process.cwd(), outputFileName);

 try {
  console.log('[Orchestrator] Rendering chapter frames to disk via headless WebGL + FFmpeg...');
  const result = await renderChapterToDisk(outputPath, 30, 5); // 5-second test render
  console.log(`[Orchestrator] Build successful! Video compiled at: ${result}`);
 } catch (error) {
  console.error('[Orchestrator] Build failed:', error);
 }
}

runOrchestrator();
```

How to Run Your Engine:

1. Ensure **FFmpeg** is installed and accessible in your system's environment PATH.
2. Install your project packages:
   Bash
   ```
   npm install
   ```
3. Run the orchestrator script to compile your first retro chapter render:
   Bash
   ```
   npm start
   ```